# Regularization & Model Selection

## BSAD 8310: Business Forecasting — Lecture 6

Department of Economics

University of Nebraska at Omaha

August 24, 2026

## The Problem: Too Many Predictors

Feature engineering gives us more predictors than we can safely estimate. Something has to decide which ones survive.

OLS minimizes $\sum_i (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2$. A weekly model with lags, calendar dummies, promotions, and macro series can easily carry $k = 60$ predictors on $n = 150$ observations — OLS will fit that sample beautifully and forecast badly.

**What goes wrong:**

- Variance of $\hat{\boldsymbol{\beta}}$ grows with $k$
- Correlated predictors give unstable, wildly large coefficients with opposing signs
- When $k > n$ there is no unique solution at all

**Regularization:** add a penalty on coefficient size,

$$\underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \underbrace{\lambda \cdot \text{Penalty}(\boldsymbol{\beta})}_{\text{complexity}}$$

Accept a little bias to buy a large reduction in variance.

Before shrinkage, the standard approach was to *count* predictors rather than shrink them (Guyon and Elisseeff 2003).

**Best subset:**

- Fit all $2^k$ models, pick the best
- Exact — hopeless past $k \approx 30$

**Stepwise:**

- Forward: add the best predictor, repeat
- Backward: start full, drop the weakest, repeat
- Greedy — can miss the best combination

**Chosen by:** adjusted $R^2$, AIC, AICc, BIC, or CV.

Subset selection is **discrete** — in or out. Small data changes flip predictors in and out, so the selected model is itself high-variance.

This is the approach in *FPP* §7.5. Shrinkage replaces the in-or-out decision with a continuous dial — more stable, and easier to tune.

## Shrinkage Methods

Keep every predictor, but pull its coefficient toward zero.

1970

$$\hat{\boldsymbol{\beta}}^{\text{ridge}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \; + \; \lambda \sum_{j=1}^{k} \beta_j^2$$

**Behavior:**

- Shrinks all coefficients, none exactly to zero
- $\lambda = 0$ is OLS; $\lambda \to \infty$ sends all $\beta_j \to 0$
- Handles correlated predictors gracefully — splits the coefficient between them

**Use when** you believe most predictors carry a little signal and you want to keep them all.

**Standardize first.** The penalty is not scale-invariant — dollars and thousands of dollars are penalized differently.

1996

$$\hat{\boldsymbol{\beta}}^{\text{lasso}} = \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \; + \; \lambda \sum_{j=1}^{k} |\beta_j|$$

**Behavior:**

- Sets some coefficients **exactly** to zero
- Selection and estimation in one step
- Yields a sparse, readable model — valuable when you must explain the forecast

**Use when** you believe only a handful of predictors genuinely matter and you want the model to tell you which.

Among correlated predictors LASSO keeps *one* arbitrarily and zeros the rest. Do not read the survivor as "the important one."

Both solve "minimize squared error subject to a budget on coefficient size." The shape of the budget is what differs.

**Ridge:** budget $\sum_j \beta_j^2 \leq t$ — a **circle**.

- Smooth boundary with no corners
- The contours of the squared-error surface almost never touch it on an axis
- So coefficients get small, but not zero

**LASSO:** budget $\sum_j |\beta_j| \leq t$ — a **diamond**.

- Corners sit exactly on the axes
- Contours are very likely to first touch a corner
- Touching a corner means $\beta_j = 0$ for that predictor

The sparsity of LASSO is a geometric accident of the $\ell_1$ ball having corners — not a statistical test of significance. A zeroed coefficient means "not worth its budget here," not "proven irrelevant." See James et al. (2023) §6.2 for the full picture.

**2005**

$$\sum_{i=1}^{n}(y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 \;+\; \lambda\left[\alpha\sum_j|\beta_j| + (1-\alpha)\sum_j\beta_j^2\right]$$

$\lambda$ sets the total penalty; $\alpha \in [0,1]$ sets the mix ($\alpha = 1$ is LASSO, $\alpha = 0$ is Ridge).

**Why the mix helps:**

- Keeps LASSO's ability to zero out predictors
- Adds Ridge's **grouping effect**: correlated predictors enter or leave together rather than one arbitrarily surviving

**Forecasting default.** Lagged predictors are correlated almost by construction — $y_{t-1}$ and $y_{t-2}$ move together. Elastic Net is usually the safer starting point over pure LASSO.

## Choosing the Penalty

$\lambda$ is not estimated from the model. It is tuned — and how you tune it decides whether the model generalizes.

Fit the model across a grid of $\lambda$ values and trace each coefficient.

**Reading the path, left to right:**

- At small $\lambda$: coefficients near their OLS values, model is dense and unstable
- As $\lambda$ grows: coefficients shrink; under LASSO they drop to zero one by one
- At large $\lambda$: only the most robust predictors survive
- At $\lambda \to \infty$: intercept-only, the mean forecast

The **order** in which predictors leave is informative: the last survivors are the ones that hold up under the most pressure. Report this ordering — it is far more stable than a single model's coefficient table.

**Procedure:**

1. Define a grid of $\lambda$, usually log-spaced
2. For each $\lambda$, run walk-forward CV and record mean validation RMSE
3. Pick the $\lambda$ minimizing CV error
4. Refit on all training data at that $\lambda$

**The one-standard-error rule:** choose the largest $\lambda$ whose CV error is within one standard error of the minimum — a simpler model at essentially the same accuracy.

**Never use `KFold` here.** Random folds put future observations in the training set and past ones in validation. The CV error will look excellent and the forecast will fail (Bergmeir et al. 2018).

Use `TimeSeriesSplit`, exactly as in Lecture 1's walk-forward evaluation.

## In Practice

Standardize, tune on a time-aware split, and read the path.

```
from sklearn.linear_model import ElasticNetCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import
TimeSeriesSplit
model = make_pipeline(
StandardScaler(),
ElasticNetCV(
l1_ratio=[.1, .5, .7, .9, .95, 1],
cv=TimeSeriesSplit(n_splits=5),
random_state=42))
model.fit(X_train, y_train)
```

**Notes:**

- `make_pipeline` keeps scaling *inside* each CV fold — scaling on the full sample leaks information
- `l1_ratio` is $\alpha$: the Ridge/LASSO mix
- `...CV` variants tune $\lambda$; plain `Ridge`/`Lasso` take it fixed
- `lasso_path()` returns the full path

`sklearn` calls the penalty `alpha`, not `lambda`.

## Key Takeaways and Roadmap

Shrinkage is the linear-model counterpart to what trees do by pruning.

| Method | Penalty | Zeros out? | Best when |
|---|---|---|---|
| OLS | none | no | $k \ll n$, predictors uncorrelated |
| Stepwise | discrete | yes | Few candidate predictors, need a story |
| Ridge | $\ell_2$ | no | Many predictors, all mildly useful |
| LASSO | $\ell_1$ | yes | Few predictors truly matter |
| Elastic Net | both | yes | Many *correlated* predictors |

**Connection to Lecture 5.** Trees and shrinkage solve the same problem in different geometries. XGBoost's `reg_lambda` is literally a Ridge penalty on leaf weights, and its `gamma` penalizes leaf count the way LASSO penalizes nonzero coefficients. Regularization is not a linear-model topic — it is how every flexible model is kept honest.

With many predictors OLS overfits: coefficient variance grows with $k$, and with $k > n$ there is no unique solution at all.

**Subset selection** counts predictors in or out. It is discrete, greedy, and unstable — small data changes flip the selected model.

**Shrinkage** replaces that switch with a dial. Ridge ($\ell_2$) shrinks everything smoothly; LASSO ($\ell_1$) shrinks and zeros out; Elastic Net does both and keeps correlated predictors together.

**Standardize first**, inside each CV fold — none of these penalties are scale-invariant. $\lambda$ **is tuned, not estimated**: use `TimeSeriesSplit`, never random $K$-fold, which leaks the future.

The **regularization path** — the order in which predictors drop out — is a more stable finding than any single coefficient table.

*Next: Introduction to Neural Networks (PyTorch) — where these same penalties reappear as weight decay and dropout.*

- Bergmeir, Christoph, Rob J. Hyndman, and Bonsoo Koo (2018). "A Note on the Validity of Cross-Validation for Evaluating Autoregressive Time Series Prediction". In: *Computational Statistics & Data Analysis* 120, pp. 70–83.
- Guyon, Isabelle and André Elisseeff (2003). "An Introduction to Variable and Feature Selection". In: *Journal of Machine Learning Research* 3, pp. 1157–1182.
- Hoerl, Arthur E. and Robert W. Kennard (1970). "Ridge Regression: Biased Estimation for Nonorthogonal Problems". In: *Technometrics* 12.1, pp. 55–67.
- James, Gareth et al. (2023). *An Introduction to Statistical Learning with Applications in Python*. 1st. New York: Springer. URL: https://www.statlearning.com/.
- Tibshirani, Robert (1996). "Regression Shrinkage and Selection via the Lasso". In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.
- Zou, Hui and Trevor Hastie (2005). "Regularization and Variable Selection via the Elastic Net". In: *Journal of the Royal Statistical Society: Series B* 67.2, pp. 301–320.